Centro Universitário de Excelência
Sistemas de Informação

API MecâniQA – Gestão de Peças e Serviços

Autor: Andrey Ramos Ferraz
Arthur Prates
Lucas Peixoto Soares Araujo
Rhuan Cassius Brito Silva

Vitória da Conquista, 2026

Agenda

Apresentar o desenvolvimento da API MecâniQA, destacando modelagem, repositórios Singleton, controladores REST e regras de negócio.

1. Modelagem das Entidades :
Visão geral das classes Peças e Serviços.

2. Repositórios Singleton:
Instância única para armazenar dados em memória.

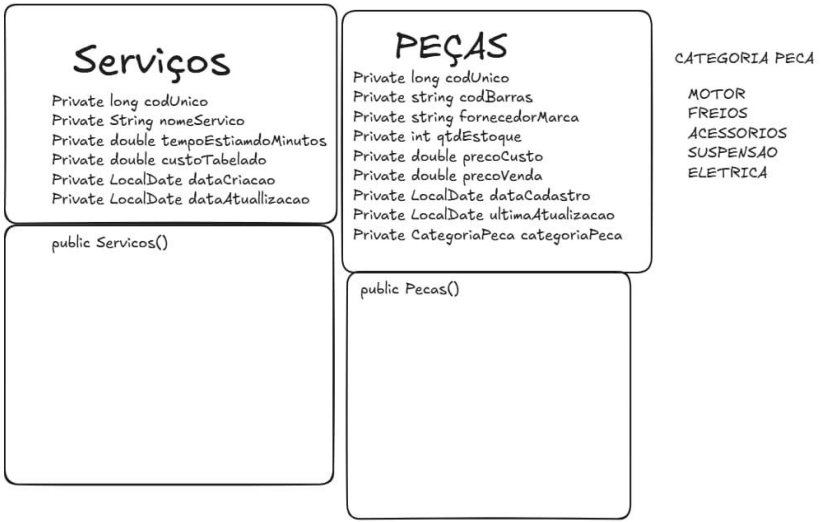
3. Controladores REST :
Mapeamento das rotas e conexão com os repositórios.

4. Regras de Negócio:
CRUD completo: cadastrar, listar, atualizar e deletar.

5. Respostas HTTP:
Status Semânticos

Modelagem das Entidades

- Atributos obrigatórios
- Atributos opcionais
- Datas automáticas
- Código único gerado por contador static



Enum CategoriaPeca

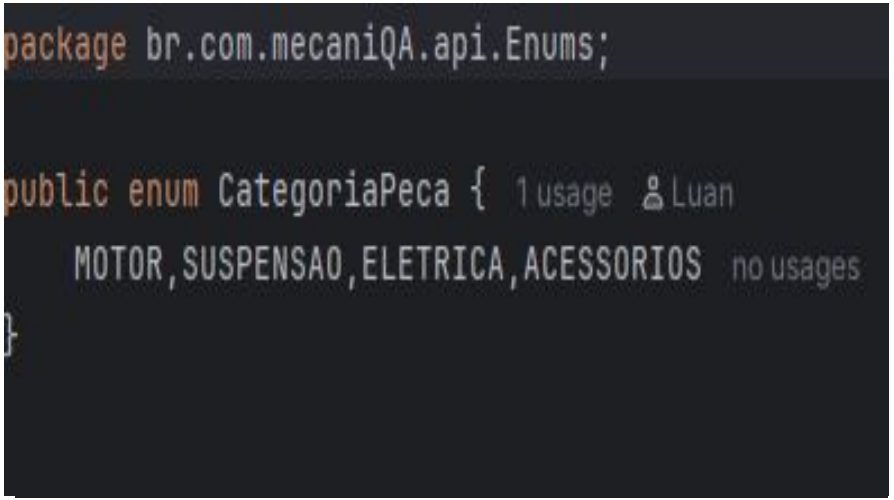
- Itens para discussão:

Peça

- Código único (contador static)
- Informações comerciais (cód. barras, marca, preços)
- Estoque
- Categoria (Enum)
- Datas automáticas
- Campos opcionais (tamanho, cor)

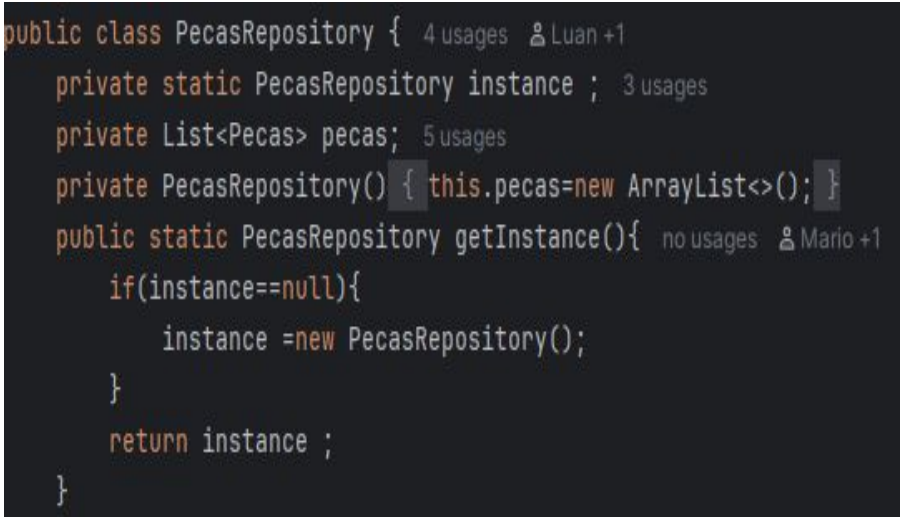
Serviço

- Código único (contador static)
- Nome, tempo estimado e custo
- Datas automáticas



Repositórios Singleton

- Itens para discussão:
  - Por que não usamos JPA
  - Por que o repositório é `static`
  - Por que o construtor é `private`
  - Como o `getInstance()` garante uma única instância
  - Como a lista interna mantém os dados enquanto o servidor roda



```
public class PecasRepository {  4 usages  Luan +1
    private static PecasRepository instance ;  3 usages
    private List<Pecas> pecas;  5 usages
    private PecasRepository() { this.pecas=new ArrayList<>(); }
    public static PecasRepository getInstance(){  no usages  Mario +1
        if(instance==null){
            instance =new PecasRepository();
        }
        return instance ;
    }
```

Controladores REST

- Itens para discussão:
  - Estrutura das rotas /api/pecas e /api/servicos
  - Uso das anotações Spring Boot (@RestController, @GetMapping, @PostMapping, etc.)
  - Fluxo das requisições: entrada → validação → chamada ao repositório → resposta HTTP
  - Tratamento de erros (404, 400)

```java
@PostMapping("/cadastrar") no usages  Mario
public ResponseEntity<Pecas> cadastrarPeca(@RequestBody Pecas pecas){
    System.out.println(pecas.getNome()+" "+pecas.getFornecedorMarca());
    Pecas novaPeca =  pecasRepository.cadastrar(pecas);
    return ResponseEntity.status(HttpStatus.CREATED).body(novaPeca);
}

@DeleteMapping("/{id}") no usages  Mario
public ResponseEntity<Pecas> deletarPeca(@PathVariable Long id){
    Pecas pecas = pecasRepository.deletarPeca(id);
    if(pecas==null){
        return ResponseEntity.notFound().build();
    }
    return   ResponseEntity.noContent().build();
}

@GetMapping("/{id}") no usages  Mario
public ResponseEntity<Pecas> buscarPorCodigo(@PathVariable Long id){
    Pecas peca =pecasRepository.buscarPorCodigo(id);
    if(peca==null){
        return   ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(peca);
}
```

Rotas e Verbos HTTP

- Itens para discussão:
  - Estrutura das rotas REST
  - Uso dos verbos HTTP
  - Relação entre ação e resposta da API
  - GET → consultar recursos
  - POST → criar recursos
  - PUT → atualizar recursos
  - DELETE → remover recursos

```java
    @GetMapping  no usages  Mario
    public List<Servicos> listarServicos(){
        return servicosRepository.listarTodosServicos();
    }

    @PostMapping("/cadastar")  no usages  Mario
    public ResponseEntity<Servicos> cadastrarServico(@RequestBody Servicos servicos){
        return ResponseEntity.status(HttpStatus.CREATED).body(servicosRepository.cadastrarServico(servicos));
    }

    @PutMapping("/{id}")  no usages  Mario
    public ResponseEntity<Servicos> atulizarServico(@PathVariable Long id,@RequestBody Servicos servicos){
        Servicos servicos1= servicosRepository.atualizarServico(id, servicos);
        if(servicos1==null){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok().build();
    }

    @DeleteMapping("/{id}")  no usages  Mario
    public ResponseEntity<Servicos> deletarServico(@PathVariable Long id){
        Servicos servicos =servicosRepository.deletarServico(id);
        if(servicos==null){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.noContent().build();
    }
}
```

Regras de Negócio

- Itens para discussão:
  - Cadastro de Peça
  - Listagem de Peças
  - Atualização de Peça
  - Exclusão de Peça
  - Cadastro de Serviço
  - Listagem de Serviços
  - Atualização de Serviço
  - Exclusão de Serviço

```
    @PostMapping("/cadastar")  no usages  Mario
    public ResponseEntity<Servicos> cadastrarServico(@RequestBody Servicos servicos){
        return ResponseEntity.status(HttpStatus.CREATED).body(servicosRepository.cadastrarServico(servicos));
    }

    @PutMapping("/{id}")  no usages  Mario
    public ResponseEntity<Servicos> atulizarServico(@PathVariable Long id,@RequestBody Servicos servicos){
        Servicos servicos1= servicosRepository.atualizarServico(id, servicos);
        if(servicos1==null){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok().build();
    }

    @DeleteMapping("/{id}")  no usages  Mario
    public ResponseEntity<Servicos> deletarServico(@PathVariable Long id){
        Servicos servicos =servicosRepository.deletarServico(id);
        if(servicos==null){
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.noContent().build();
    }
}
```

Respostas HTTP

- Itens para discussão:
  - **201 Created** – quando um recurso é criado
  - **200 OK** – listagens e atualizações bem-sucedidas
  - **204 No Content** – deleção bem-sucedida
  - **404 Not Found** – ID não encontrado
  - **400 Bad Request** – dados inválidos (se implementado)

Considerações finais

Pontos de Interesse:

- Aprendizado do padrão Singleton
- Entendimento profundo das anotações Spring Boot
- Construção de CRUD completo sem JPA
- Importância da semântica dos status HTTP
- Organização do código e boas práticas REST

Referências

- SPRING FRAMEWORK. **Spring Boot Reference Documentation**. [S.l.]: Pivotal Software, 2024. Disponível em: https://docs.spring.io/spring-boot/docs/current/reference/html/
- ORACLE. **Java Platform, Standard Edition Documentation**. [S.l.]: Oracle Corporation, 2024. Disponível em: https://docs.oracle.com/javase/
- Spring Framework. (2024). *Spring Boot reference documentation*. Pivotal Software. https://docs.spring.io/spring-boot/docs/current/reference/html/
- W3C. **Web Services Glossary**. 2004. Disponível em: https://www.w3.org/TR/ws-gloss/